Chapter route

How can services remain understandable and recoverable without shared memory, synchronized clocks, or failure-free nodes?

Prerequisites are network stacks, RPC, processes, and filesystem interfaces.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Learning objectives

- Compare network and distributed operating systems.
- Analyze naming, routing, RPC, and failure semantics.
- Check DFS cache consistency and availability.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Advantages

Sharing, speedup, and reliability require cost/failure assumptions. Nodes communicate by messages without a perfect common clock.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Message passing and its limits

Each node has local memory/OS and exchanges messages. Local/remote are relative to the requester. Resource sharing, computation speedup, reliability, and communication motivate distribution without following automatically from it.

Of a 100 s job,20 s is serial,80 s divides across four nodes, and coordination costs 5 s. $T4=20+80/4+5=45$ s, speedup$100/45\approx2.222$. Ignoring coordination, the infinite-node Amdahl bound is$100/20=5$. More nodes cannot remove serial work and can increase coordination cost.

Three independently failing interchangeable read replicas each have $p=.01$ over the interval. With reliable networking/metadata and any replica sufficient, unreadability is $p^3=10^{-6}$. Shared power/rack or required metadata-leader survival violates this independence model. Replica count is not a reliability proof.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Network-based operating systems

Remote login, file transfer, and DFS expose different sharing semantics. Data, computation, and process migration move different state.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Transparency and machine boundaries

Network OSs expose remote hosts/copies explicitly. Distributed OSs/middleware can unify naming, access, and migration. Transparency packages location/invocation and failure semantics; it does not remove physical latency/failure.

Remote login executes a remote session; transfer creates another file copy; DFS provides shared remote-file behavior. SSH/SFTP still need host authentication/user authorization; ordinary FTP is not inherently encrypted. Anonymous services need constrained namespaces/operations. Naming, identity mapping, network faults, and conflicting copies require rules.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Moving data or computation

For1 GiB input,1 MiB result,100 MiB/s throughput, and20 ms RTT, moving all input costs at least1024/100+.020=10.260 s before local compute. Sending computation to a remote CPU taking.500 s and returning 1 MiB takes.500+.010+.020=.530 s, with negligible request. Include CPU queues, startup, serialization, and output size before generalizing.

Whole-file caching pays upfront for later local access. On-demand caching transfers requested blocks. Reading only1 MiB makes a whole1 GiB transfer waste1023 MiB; reading all blocks through unpipelined RTT-bound RPCs can instead favor bulk transfer.

Process migration moves/reconstructs PC, registers, memory, open files/sockets, and environment. Sending code for fresh remote execution is computation placement, not migration of running state. Cross-ISA/ABI execution needs compatibility rather than copying register bits blindly.

Network structure

LAN/WAN are not fixed throughput rankings. Separate serialization, propagation, queueing, and processing. BDP/window bounds govern in-flight data.

Scope and latency components

LANs commonly cover smaller administrative/geographic areas; WANs connect sites. Capacity/latency depend on links, distance, queues, load. Switches, routers, and APs have different roles. Switched Ethernet is not universally one shared collision bus; a congested LAN can underperform a high-capacity WAN.

Latency≈L/R+propagation+queue+processing. A 1500 B packet on1 Gbit/s serializes in 12 µs. One-way5 ms propagation dominates. A tenfold rate increase reduces serialization to1.2 µs without dividing propagation by ten.

100 Mbit/s equals12.5 decimal MB/s before headers; MiB/s uses2^{20} B. Wireless nominal rates differ from goodput. These are explicit models rather than current product maxima.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Bandwidth-delay product

100 Mbit/s and40 ms RTT give BDP=4×10^6 bits=500000 B≈488.28 KiB. A 64 KiB outstanding-data limit bounds throughput at 65536・8/.040=13.1072 Mbit/s despite100 Mbit/s capacity.

This assumes no loss, fixed RTT, and no other bottleneck. TCP congestion/receive windows, ACK behavior, slow start, retransmission further constrain results. Send buffers remove at most one bound, not congestion/server CPU. A 16 B RPC can be dominated by RTT rather than serialization, explaining the importance of placement/cache.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Communication structure

DNS names, IP addresses, ports, and stable object identity differ. Routing and connection semantics require explicit state/order assumptions.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Names, addresses, ports, and objects

Store.lab.test is a domain name, possibly resolving to198.51.100.20 or IPv6. IP addresses are routable endpoint addresses, not users or permanent machine identities. Service identity also involves protocol/port/application naming. A TCP connection uses source/destination addresses/ports and protocol; a port is not a PID.

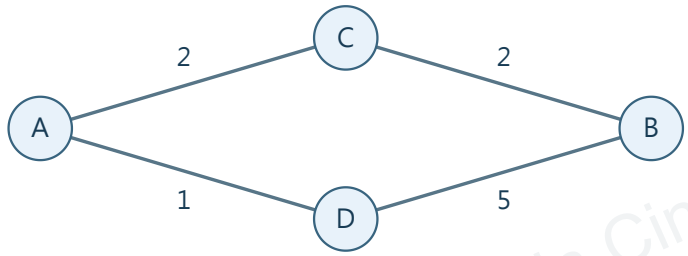
DNS uses resolver, queries, caches, and delegation rather than a server per label. Assume root delegates test and test delegates lab.test. On a cache miss, resolver follows known-root referrals to an authoritative A record. TTL bounds cache lifetime without immediate global invalidation.

At t0, an A record=.20 with TTL600 can remain cached at t100 with 500 s left. An authoritative change to.30 at t200 need not replace it until expiry at 600. Only successful new resolution obtains a checked current value. DNS integrity and endpoint authentication require corresponding mechanisms.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Routing on a fixed graph

One nonnegative weighted graph: every distance update



From A: D=1, C=2, B=∞

Settle D: B=1+5=6

Settle C: B=min(6,2+2)=4

Settle B: A → C → B, cost4

If C–B fails, recompute A→D→B, cost6. Dynamic protocols need detection/convergence time.

Edges are A–C2, C–B2, A–D1, D–B5. Dijkstra starts dD1, dC2, dB∞. Settling D gives dB6; settling C lowers it to4; settling B yields A→C→B, cost4. Losing C–B gives A→D→B, cost6 after recomputation.

Routing on a fixed graph (continued)

Static routing uses configured/admin rules; virtual circuits select session paths; dynamic protocols update metrics/reachability. Dynamic does not mean every packet chooses the emptiest link or converges without loops/drops. A path preserves order only with suitable link/queue behavior; transport supplies end-to-end stream ordering.

Control plane computes routes; data plane forwards from tables. SDN can separate them but still needs consistent updates, failure handling, and protected management.

Switching and connections

Frames, packets, and datagrams have layer-specific headers/limits and can vary in length. Application messages can span packets or share a TCP read. TCP reserves no dedicated physical circuit; applications can implement reliability over UDP.

Circuit switching reserves path resources; message switching stores/forwards whole messages; packet switching pipelines fragments with statistical multiplexing. Across two1 Mbit/s links, ignoring headers/propagation/queues, a 4000 B message takes 64 ms. Four1000 B packets each take8 ms/hop; pipeline completion is(2+4−1) • 8=40 ms.

With40 B per-packet headers, it becomes5 • 1040 • 8/10^6=41.6 ms. One40 B whole-message header gives64.64 ms. These are store-and-forward assumptions, not all cut-through/circuit networks.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Communication protocols

OSI describes responsibilities, not fixed hardware assignments. Payload, headers, MTU, and wire byte-times use different denominators.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Layer responsibilities

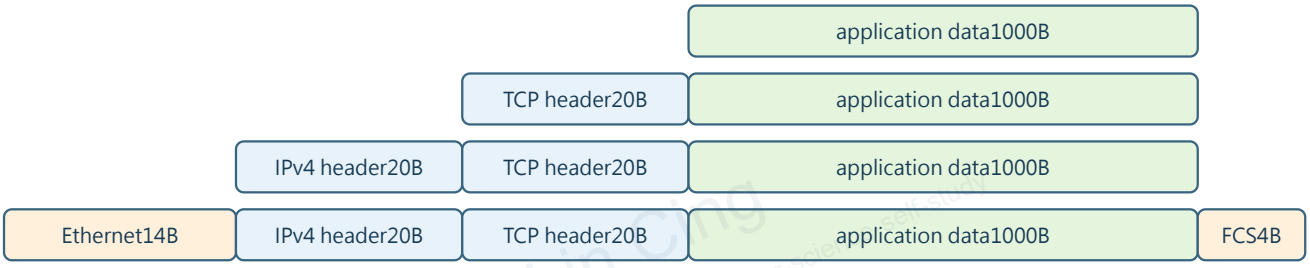
Physical carries signals/bits; link carries frames; network addresses/forwards; transport serves process endpoints; session manages dialogue; presentation encodes representations; application defines service operations. TCP/IP often places session/presentation in libraries/applications. Fewer named layers does not prove efficiency.

TCP offers ordered reliable bytes and flow/congestion control without message boundaries. UDP preserves datagrams without equivalent built-in delivery/order. IP is best effort. NIC offload can move checksums/segmentation into hardware, so no universal three-hardware/four-software split exists.

Serialization specifies lengths, byte order, encoding, versions. Raw C structs with padding/pointers are not portable protocols.

Encapsulation length accounting

Each layer adds headers/trailers; account for every length



Widths are not to scale. IP total1040 B; MAC frame1058 B. Preamble/SFD and IFG are separate.

1000 B application data plus20 B TCP gives1020 B segment;20 B IPv4 gives1040 B IP total. EthernetII14 B header+1040+4 FCS=1058 B. Add8 B preamble/SFD and specified12 byte-times IFG for 1078 wire byte-times.

Encapsulation length accounting (continued)

WithIPv4 MTU1500, this fits one frame. MSS subtracts TCP header from available IP payload;1500 is not all application data. VLAN/options/tunnels/security change accounting. TLS endpoints are not every forwarding router. A lower checksum detects accidental corruption rather than authenticating malicious changes with a secret key.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

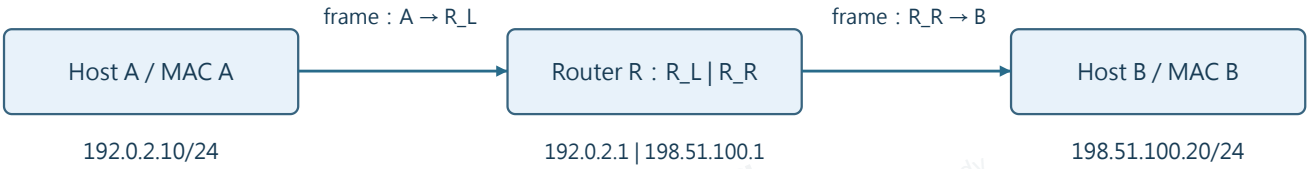
TCP/IP example

ARP resolves the next hop; each hop rebuilds link addresses. IPv4 TTL/checksum change while IP endpoints remain in the no-NAT model.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Two Ethernet LANs

Across LANs without NAT: each hop uses different MAC addresses



A ARPs next-hop 192.0.2.1, not remote B on the left LAN.

IP stays192.0.2.10 → 198.51.100.20. Router decrements TTL64→63; recompute checksum/FCS.

No NAT/tunnel. A=192.0.2.10/24, MAC A. Router left=192.0.2.1, MAC R_L; right=198.51.100.1, MAC R_R. B=198.51.100.20/24, MAC B. DNS gives B; /24 comparison selects next-hop192.0.2.1.

Two Ethernet LANs (continued)

Without cached R_L, A broadcasts ARP for 192.0.2.1 on its LAN. R answers. Frame1 has srcA, dstR_L, IP srcA, dstB. A does not ARP remote B directly. Router looks up route, decrementsTTL64→63, updates IPv4 checksum, resolves B on the right LAN, and sends frame2 srcR_R, dstB with unchanged A→B IP endpoints.

B decapsulates and demultiplexes by transport port/connection. Routers normally do not forward ARP broadcasts across L2 domains; ARP replies do not cryptographically prove identity.

Header changes and minimum frames

Each hop rebuilds Ethernet addresses/FCS. IPv4 TTL/checksum change; source/destination IP, TCP endpoints, payload remain under the stated exclusions. Saying only Ethernet changes is false. TTL reaching zero prevents normal forwarding; error reports can themselves be lost/filtered.

Untagged EthernetII has6 B destination,6 B source,2 B EtherType,46–1500 B data/pad,4 B FCS, minimum64 B from destination through FCS. Padding is not included in IP total length; preamble/SFD is outside that frame denominator.

A 4 B TCP payload plus20+20 headers is44 B IP, requiring2 B pad. Frame14+46+4=64 B. Add8 B preamble/SFD and96-bit IFG=12 byte-times to84. At100 Mbit/s, it occupies6.72 µs. IFG is idle time, not12 bytes appended to IP; high-speed PHY tolerances are outside this model.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

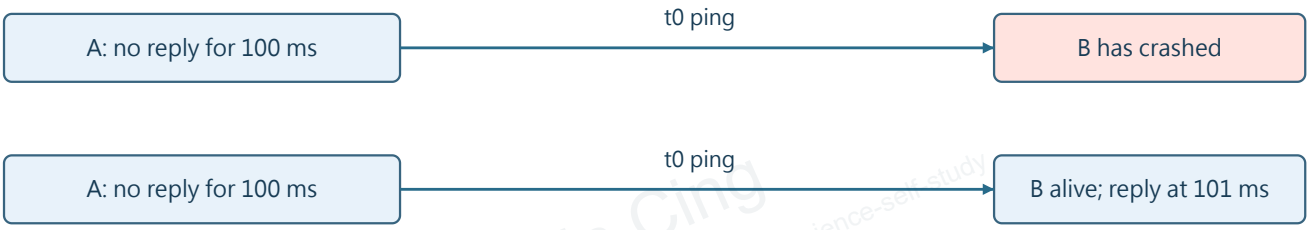
Robustness

Unbounded delays make timeout suspicion rather than proof. Fencing, transactional deduplication, and recovery handle different partial failures.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Failure-detector indistinguishability

Same local observation, different remote states



A has the same history through 100 ms. Without delay bounds, timeout does not prove crash.

In an asynchronous system without known delay bounds, A pings at 0 and sees no reply by100 ms. B may have crashed, partitioned, lost messages, or merely run slowly. Compare B crashed with B alive but replying at 101 ms. A's local history through100 is identical. Any algorithm declaring crash solely from it errs in the second execution.

Failure-detector indistinguishability (continued)

With heartbeat intervalH1 s, max send scheduling delays.2 s, and one-way network boundδ.1 s, the largest normal arrival gap is≤H+s+δ=1.3 s, assuming the previous arrival could be immediate. Timeout1.5 provides margin under these premises. Invalidating the delay bound invalidates the no-false-suspicion guarantee.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

New leaders need old-leader fencing

A writes shared storage. Partitioned B times out and declares itself leader while A still writes, creating split brain. Heartbeats/local declarations do not guarantee exclusion.

A correct coordinator issues monotonic epochs. Storage atomically installs currentEpoch43 before new B operates, and every write checks token==43. A's42 writes fail. This requires correct issuance/installation and every write path enforcing it; partitions cannot independently increment into legitimate ownership. Confirmed hardware/access isolation can also fence old nodes.

Recovering A synchronizes committed state, membership/epoch, and unfinished requests before becoming follower. Old local data does not restore leadership. IDs/generations protect against stale callbacks/messages after restart.

Lost RPC replies and deduplication

Counter 10 receives increment(id17), commits11, and loses reply. Naive retry yields12. TCP retransmission supplies connection-byte delivery, not cross-reconnection application deduplication.

In the explicit at-most-once model, a server transaction checks durable table[id]. Existing IDs return saved results; new IDs atomically persist counter++ and table[id]=result before replying. Retrying17 returns 11. Updating counter before recording ID permits duplicate increment after crash; recording ID first permits lost effect.

The proof requires shared transaction/recovery scope, unique IDs, and retention long enough for retries. Permanent disconnection can still leave the client uncertain. External email/payment outside the transaction needs another protocol.

Fault domains and failover capacity

Two active servers each handle100/s with total load160. Losing one leaves100 and at least60/s unsustainable. Replicas being able to take over does not preserve SLO without spare capacity, shedding, or degradation.

Stateless worker retries can still duplicate effects. Stateful services additionally need logs, recovery, replication, locks. RAID addresses selected disk faults, not cross-site availability or backups of logical corruption. Recover durable committed state, reconcile in-flight operations, then serve. Missing commit reply does not imply uncommitted work.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Design issues

Transparency retains failure semantics; the narrowest resource limits scale. HDFS uses historical1.2.1 roles and explicit replica/reconstruction costs.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Observable transparency and scaling

Location transparency hides placement;migration transparency preserves naming/operations after movement;replication transparency hides copies. Failure transparency cannot indefinitely hide a nonreturning operation. User mobility also needs identity, environment, and session consistency.

Ten workers at 100/s offer1000/s, but a one-core coordinator spending2 ms/request caps the system at 500/s. An eleventh worker does not lift it. Caching, batching, or partitioning can alter the bottleneck's per-request demand. Replication/rebuild traffic consumes capacity too.

Hadoop1.2.1 HDFS NameNode manages namespace/block locations;DataNodes store/transfer blocks;client data bypasses NameNode. Large blocks, streaming, and a simplified write model favor bulk data rather than full POSIX sharing or arbitrary low-latency transactions.

HDFS payload and flow

Custom64 MiB blocks and replication3 split130 MiB into64,64,2. Logical130 yields390 MiB replicated payload, excluding metadata, checksums, padding/allocation. The last2 MiB need not transfer a full64 MiB of payload but still needs block location metadata.

Client gets block/DataNode locations from NameNode, then reads DataNodes directly; writes can use a DataNode pipeline. Heartbeats/block reports detect missing replicas and trigger re-replication with delay/network/disk cost.

Secondary NameNode assists checkpointing;it is not an automatic hot standby. Fault-domain-aware placement helps data availability, while metadata and client reachability remain required.

Distributed file systems

Separate names, FIDs, locations, and cache validity. NFSv3 stable modes, COMMIT, and verifiers define restart handling.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Names, stable identities, and locations

Location transparency hides physical placement;independence preserves names after movement. ServerA:/x exposes server location. Client mounts can hide it differently per client;global namespaces add common naming/management.

Path/course/a.txt→FID(volume7, file 12, generation 3), volume7→serverA. Moving to serverB changes only the second map. Reusing deleted file 12 must change generation or stale FIDs can identify the new file. Path→FID and volume→location caches have different invalidation conditions.

Replicated/cached location maps still need versions/coherence. Indirection does not eliminate updates. Diskless boot requires firmware/initial protocols to load kernel before DFS exists; an unloaded DFS cannot load itself.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Complete cache-cost accounting

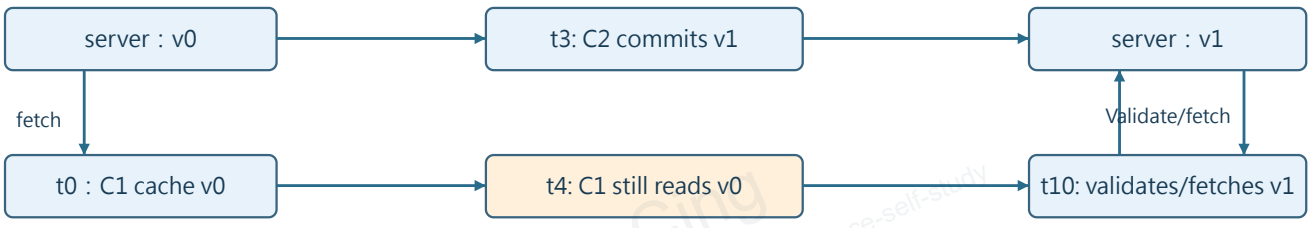
100 reads of 4 KiB at 5 ms remote cost total500 ms uncached. Ten misses at 5 plus90 hits at.1 cost59 ms and40 KiB network payload versus400 KiB. This specified workload does not prove a universal hit ratio.

Per-read 1 ms validation adds100, for 159 ms. Cache hits need not avoid network RTT. Larger cache units can improve read-ahead or waste bandwidth on a one-byte read and reduce resident-block counts at fixed capacity.

Memory cache is volatile;disk cache retains durable bytes but needs metadata recovery/freshness checks. Servers cache too;server RAM acceptance is not stable storage. Write-through/back specify propagation timing, while the next layer's return defines durability separately.

Cache consistency timelines

Server versions and client cache visibility have different timelines



Periodic checks permit staleness. Strong callback/lease models recall conflicting read rights before write completion.

Client-initiated validation:C1 reads/validatesv0 at 0;C2 completesv1 at 3;C1 validating next at 10 can still returnv0 at 4. This is not latest-completed-write visibility on each read. Open-time validation needs within-open semantics;every-access validation costs network and still needs coordination with the actual read.

Cache consistency timelines (continued)

Server-initiated coherence tracks cache/read/write rights and invalidates/recalls them, or disables cache on conflict. A strong model requires confirmation that conflicting read permissions are revoked/invalidated or their controlled leases expired before write completion. During partition, unlimited old-client reads cannot coexist with an immediate-visibility claim.

AFS/OpenAFS callbacks avoid checking every access, but do not rewrite application-owned bytes already read. Save/close and cache-manager refresh affect observations. Session, close-to-open, and stronger DFS guarantees are not synonyms.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

NFSv3 state and write stability

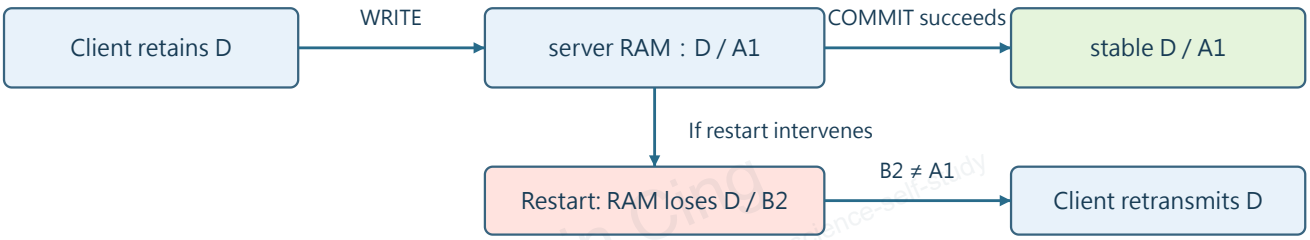
RFC1813 READ/WRITE carry filehandle, offset, count rather than relying on equivalent server OPEN sessions. Server caches/duplicate-request state and separate lock protocols can still exist. Authentication, permissions, and handle lifetime remain relevant.

WRITE requests/replies distinguish UNSTABLE, DATA_SYNC, FILE_SYNC. UNSTABLE success is not durable data. Clients requiring later COMMIT/verifier checks retain retransmittable bytes. Weak cache-consistency attributes assist change detection without proving global linearizability.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Restart after an unstable write

NFSv3: normal and restart paths after one successful WRITE



After UNSTABLE, retain retransmission data until a stable result with a matching verifier.

Client sends4 KiB D at offset 0, UNSTABLE. Server receives into RAM and replies UNSTABLE, verifierA1;client retains D. Normal COMMIT succeeds with A1, establishing protocol-stable storage and permitting release of that retransmission obligation.

Restart after an unstable write (continued)

If restart loses volatile D, new instance usesB2. A COMMIT/WRITE returningB2≠A 1 cannot certify the old data. Retransmit unconfirmed D and obtain the required stable result. COMMIT success without verifier comparison is insufficient.

This requires retained client bytes, correct server stability/verifier/range behavior, and does not give arbitrary concurrent writers application transactions. Durability, visibility, and RPC deduplication differ.

One remote operation across layers

Service name→DNS address→route→ARP→link/IP/transport→RPC ID→authorization/durable commit→client cache. Frame delivery does not prove RPC completion;TCP bytes do not prove one business execution;NFS UNSTABLE does not prove persistence.

Transparency offers a consistent interface inside stated boundaries without eliminating latency, partial failures, copies, or external effects. Insufficient observations require unknown/suspected rather than treating timeout as logical proof.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Reconstructable states and arguments

Split serial/parallel/coordination cost. Track bits/bytes, headers, RTT, windows. Draw reliability fault domains. Link RPC IDs to atomic effects;separate DFS names, identities, locations, visibility, durability. General guarantees depend on transitions:unbounded-delay timeout cannot prove crash, fencing needs resource-side epoch enforcement, and deduplication needs sufficient retention. Exercises include counterexamples, normal paths, and restart states.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

References

Silberschatz, Galvin, and Gagne,*Operating System Concepts*,9th edition, Chapter 17, printed pages 741–774. Numerical cases, figures, and exercises are original.

RFC1034 covers DNS;RFC826 ARP;RFC1813, § 3.3.7,3.3.21 NFSv3 stable writes/COMMIT/verifiers. Addresses here are documentation examples. HDFS1.2.1 architecture and users guide define historical roles/checkpointing. OpenAFS concepts supplies callback context without assumed universal chunk defaults. Cisco frame/IPG reference distinguishes frame fields from idle gap;the numerical case fixes untagged100 Mbit/s conditions.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

References and attribution

Abraham Silberschatz, Peter B. Galvin, Greg Gagne. Operating System Concepts, 9th Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study